Checksum Algorithm (8-bit) - C++ Implementation

ID - Card

Name: Srimoneyshankar Ajith
Roll_Number: 2024BCS0055
Date: 02-03-2026

Aim

To implement the 8-bit Checksum algorithm to detect two or more bit errors during message transmission from sender to receiver.

Theory

In the 8-bit Checksum algorithm:

- Data is divided into 8-bit words.
- All words are added using 1's complement addition.
- The final sum is complemented (1's complement).
- The receiver adds all received words including the checksum.
- If the result is 11111111 (255), there is no error.
- Otherwise, an error is detected.

Algorithm

Sender Side

1. Take data words (8-bit each).
2. Add them using 1's complement addition.
3. Take 1's complement of the final sum.
4. Append the checksum to the data.

Receiver Side

1. Add all received data words including the checksum.
2. If final result is 255, no error.
3. Otherwise, error detected.

C++ Program

```cpp
#include <iostream>
#include <vector>

using namespace std;

int add_checksum(int sum, int value) {
    sum += value;
    if (sum > 255) {
        sum = (sum & 255) + 1;
    }
    return sum;
}

int generate_checksum(const vector<int>& data) {
    int sum = 0;
    for (int i = 0; i < data.size(); ++i) {
        sum = add_checksum(sum, data[i]);
    }
    return (~sum) & 255;
}

bool verify_checksum(const vector<int>& received_data) {
    int sum = 0;
    for (int i = 0; i < received_data.size(); ++i) {
        sum = add_checksum(sum, received_data[i]);
    }
    return (sum == 255);
}

void print_data(const vector<int>& data) {
    for (int i = 0; i < data.size(); ++i) {
        cout << data[i];
        if (i != data.size() - 1)
            cout << ", ";
    }
    cout << endl;
}

int main() {
    // Test Case 1
    cout << "Test Case 1:\n";
    cout << "Data word → 160, 12, 171, 20\n";
    cout << "No errors in the bits\n";
```

```
    vector<int> data1 = {160, 12, 171, 20};
    int checksum1 = generate_checksum(data1);

    cout << "Generated Checksum: " << checksum1 << endl;

    data1.push_back(checksum1);

    cout << "Transmitted Data (with checksum): ";
    print_data(data1);

    if (verify_checksum(data1))
        cout << "Receiver Result: No Error Detected\n";
    else
        cout << "Receiver Result: Error Detected\n";

    cout << "\n";

    // Test Case 2
    cout << "Test Case 2:\n";
    cout << "Data word → 160, 12, 171, 20\n";
    cout << "Corrupted bits →\n";
    cout << "160 : Leftmost bit flipped\n";
    cout << "12  : 2nd bit from leftmost flipped\n";
    cout << "171 : Leftmost bit flipped\n";
    cout << "20  : 3rd bit from leftmost flipped\n";

    vector<int> data2 = {160, 12, 171, 20};
    int checksum2 = generate_checksum(data2);

    data2[0] ^= 128;
    data2[1] ^= 64;
    data2[2] ^= 128;
    data2[3] ^= 32;

    data2.push_back(checksum2);

    cout << "Received Data (with checksum): ";
    print_data(data2);

    if (verify_checksum(data2))
        cout << "Receiver Result: No Error Detected\n";
    else
        cout << "Receiver Result: Error Detected\n";

    return 0;
}
```

Output

```
■ money on Aether via ⚙ v6.3.0 at …\College\4th Semester\ICS224 ❯ g++ crc.cpp

■ money on Aether via ⚙ v6.3.0 at …\College\4th Semester\ICS224 ❯ ./a.exe
Test Case 1:
Data word -> 160, 12, 171, 20
No errors in the bits
Generated Checksum: 147
Transmitted Data (with checksum): 160, 12, 171, 20, 147
Receiver Result: No Error Detected

Test Case 2:
Data word -> 160, 12, 171, 20
Corrupted bits ->
160 : Leftmost bit flipped
12  : 2nd bit from leftmost flipped
171 : Leftmost bit flipped
20  : 3rd bit from leftmost flipped
Received Data (with checksum): 32, 76, 43, 52, 147
Receiver Result: Error Detected
```

Conclusion

The program implements the 8-bit 1's complement checksum method. It correctly detects multiple bit errors introduced during transmission.